

Logistic regression implementation

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
In [2]: from sklearn.datasets import make_classification
```

```
In [3]: #create the dataset
X,y=make_classification(n_samples=1000, n_features=10, n_classes=2, random_state=42)
```

```
In [4]: X
```

```
Out[4]: array([[ 0.96479937, -0.06644898,  0.98676805, ..., -1.2101605 ,
                -0.62807677,  1.22727382],
               [-0.91651053, -0.56639459, -1.00861409, ..., -0.98453405,
                0.36389642,  0.20947008],
               [-0.10948373, -0.43277388, -0.4576493 , ..., -0.2463834 ,
                -1.05814521, -0.29737608],
               ...,
               [ 1.67463306,  1.75493307,  1.58615382, ...,  0.69272276,
                -1.50384972,  0.22526412],
               [-0.77860873, -0.83568901, -0.19484228, ..., -0.49735437,
                2.47213818,  0.86718741],
               [ 0.24845351, -1.0034389 ,  0.36046013, ...,  0.77323999,
                0.1857344 ,  1.41641179]], shape=(1000, 10))
```

```
In [5]: #convert X to dataframe
pd.DataFrame(X)
```

```
Out[5]:
```

	0	1	2	3	4	5	6
0	0.964799	-0.066449	0.986768	-0.358079	0.997266	1.181890	-1.615679
1	-0.916511	-0.566395	-1.008614	0.831617	-1.176962	1.820544	1.752375
2	-0.109484	-0.432774	-0.457649	0.793818	-0.268646	-1.836360	1.239086
3	1.750412	2.023606	1.688159	0.006800	-1.607661	0.184741	-2.619427
4	-0.224726	-0.711303	-0.220778	0.117124	1.536061	0.597538	0.348645
...	...	...	...	...	...	...	...
995	-1.367638	1.462255	-1.154918	-0.290454	-0.413424	0.032396	1.545490
996	-1.514876	-3.221016	-1.300744	0.395599	-0.527994	1.353069	1.777506
997	1.674633	1.754933	1.586154	0.018402	-1.514470	0.321593	-2.417694
998	-0.778609	-0.835689	-0.194842	1.097220	0.180071	-0.272933	-0.533188
999	0.248454	-1.003439	0.360460	-0.331352	1.389764	-1.345210	-0.740875

1000 rows × 10 columns

```
In [6]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.30,random_
```

```
In [7]: #model training
from sklearn.linear_model import LogisticRegression
logistic=LogisticRegression()
logistic.fit(X_train,y_train)
y_pred=logistic.predict(X_test)
print(y_pred)
```

```
[0 1 0 1 0 1 0 0 0 0 0 1 0 1 0 0 1 1 1 1 0 0 1 0 0 1 0 0 1 1 1 1 0 0 0 0 1
1 1 1 0 1 1 0 0 0 1 1 1 1 0 1 0 0 1 0 1 0 1 0 0 1 1 1 0 0 1 1 1 1 1 1 0
1 0 0 1 0 1 0 0 1 0 1 0 0 0 0 1 1 1 1 1 1 1 0 0 1 0 1 0 1 0 0 1 0 1 1 1 1
1 1 1 1 0 0 1 0 0 1 1 0 1 1 1 1 1 1 1 1 0 1 1 1 1 0 0 0 1 1 0 0 0 0 0 1 0
0 0 1 0 0 1 0 0 1 0 1 1 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 0 0 1 1 1 1 1 1
0 0 0 0 1 0 0 0 0 1 0 0 1 1 1 0 1 0 0 0 1 1 1 1 1 0 0 0 0 0 1 0 1 0 1 1 0
0 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 1 1 0 1 0 1 1 0 1 0 1 1 1 0 0 1 1 1 1 1
0 1 0 1 1 0 0 0 1 1 0 1 1 0 0 1 0 0 0 0 1 1 0 1 0 1 1 1 0 0 1 0 1 1 0 1 1
1 1 1 0]
```

```
In [8]: logistic.predict_proba(X_test)
```

```
Out[8]: array([[7.74477909e-01, 2.25522091e-01],
               [3.36684957e-02, 9.66331504e-01],
               [6.70682154e-01, 3.29317846e-01],
               [7.98668032e-02, 9.20133197e-01],
               [9.76616650e-01, 2.33833501e-02],
               [4.13572804e-02, 9.58642720e-01],
               [9.79028767e-01, 2.09712329e-02],
               [9.59367261e-01, 4.06327393e-02],
               [8.08520049e-01, 1.91479951e-01],
               [6.84954318e-01, 3.15045682e-01],
               [9.13669448e-01, 8.63305524e-02],
               [2.63597018e-01, 7.36402982e-01],
               [5.25844192e-01, 4.74155808e-01],
               [2.11912354e-01, 7.88087646e-01],
               [7.93592056e-01, 2.06407944e-01],
               [9.46621678e-01, 5.33783219e-02],
               [2.62957933e-02, 9.73704207e-01],
               [3.24212617e-01, 6.75787383e-01],
               [3.14803584e-01, 6.85196416e-01],
               [2.04956217e-01, 7.95043783e-01],
               [5.04587065e-01, 4.95412935e-01],
               [9.66703630e-01, 3.32963696e-02],
               [2.00514796e-01, 7.99485204e-01],
               [7.77891908e-01, 2.22108092e-01],
               [8.72986315e-01, 1.27013685e-01],
               [4.01990726e-02, 9.59800927e-01],
               [9.21218522e-01, 7.87814776e-02],
               [8.14302359e-01, 1.85697641e-01],
               [4.84323961e-01, 5.15676039e-01],
               [3.45979014e-01, 6.54020986e-01],
               [6.82517287e-02, 9.31748271e-01],
               [5.40154632e-04, 9.99459845e-01],
               [9.69329756e-01, 3.06702442e-02],
               [9.72909425e-01, 2.70905746e-02],
               [9.42369499e-01, 5.76305006e-02],
               [5.19300818e-01, 4.80699182e-01],
               [8.46478994e-02, 9.15352101e-01],
               [3.25968944e-01, 6.74031056e-01],
               [4.76883185e-01, 5.23116815e-01],
               [1.53788412e-04, 9.99846212e-01],
               [9.15879079e-01, 8.41209206e-02],
               [8.64908414e-02, 9.13509159e-01],
               [4.79528887e-01, 5.20471113e-01],
               [9.35681383e-01, 6.43186173e-02],
               [9.77892918e-01, 2.21070816e-02],
               [5.85645655e-01, 4.14354345e-01],
               [1.57842356e-02, 9.84215764e-01],
               [7.62135212e-02, 9.23786479e-01],
               [1.34855065e-02, 9.86514493e-01],
               [5.30445112e-02, 9.46955489e-01],
               [8.06446503e-01, 1.93553497e-01],
               [4.80600460e-03, 9.95193995e-01],
               [5.04847224e-01, 4.95152776e-01],
               [8.68420588e-01, 1.31579412e-01],
               [8.92652368e-03, 9.91073476e-01],
               [5.92487141e-01, 4.07512859e-01],
               [1.57430997e-01, 8.42569003e-01],
               [9.76375749e-01, 2.36242514e-02],
               [9.85861892e-02, 9.01413811e-01],
               [9.04177979e-01, 9.58220211e-02],
```

[6.72480581e-03, 9.93275194e-01],
[6.94837167e-01, 3.05162833e-01],
[7.57553517e-01, 2.42446483e-01],
[3.10057665e-02, 9.68994234e-01],
[3.06669056e-01, 6.93330944e-01],
[1.74765000e-01, 8.25235000e-01],
[8.36522765e-01, 1.63477235e-01],
[8.97499950e-01, 1.02500050e-01],
[3.36207606e-01, 6.63792394e-01],
[1.63589641e-01, 8.36410359e-01],
[3.22758711e-01, 6.77241289e-01],
[4.48390227e-02, 9.55160977e-01],
[2.14643172e-04, 9.99785357e-01],
[9.68626181e-01, 3.13738187e-02],
[1.85275122e-01, 8.14724878e-01],
[8.76090291e-01, 1.23909709e-01],
[9.74552911e-01, 2.54470888e-02],
[1.69192536e-03, 9.98308075e-01],
[9.42494179e-01, 5.75058208e-02],
[1.66988763e-01, 8.33011237e-01],
[7.93678282e-01, 2.06321718e-01],
[9.40836049e-01, 5.91639514e-02],
[1.01610448e-01, 8.98389552e-01],
[8.74075354e-01, 1.25924646e-01],
[2.28534077e-04, 9.99771466e-01],
[8.78526575e-01, 1.21473425e-01],
[8.44941860e-01, 1.55058140e-01],
[8.53358212e-01, 1.46641788e-01],
[8.69056654e-01, 1.30943346e-01],
[1.28582007e-01, 8.71417993e-01],
[2.73397191e-03, 9.97266028e-01],
[2.14400967e-02, 9.78559903e-01],
[1.08228331e-01, 8.91771669e-01],
[2.63727024e-01, 7.36272976e-01],
[5.77997736e-02, 9.42200226e-01],
[2.16038259e-01, 7.83961741e-01],
[9.32819141e-01, 6.71808586e-02],
[9.41557701e-01, 5.84422990e-02],
[4.08768304e-01, 5.91231696e-01],
[9.17168952e-01, 8.28310478e-02],
[4.74767472e-03, 9.95252325e-01],
[9.38736508e-01, 6.12634915e-02],
[1.56652749e-02, 9.84334725e-01],
[9.97834411e-01, 2.16558917e-03],
[9.91498786e-01, 8.50121430e-03],
[2.95803470e-01, 7.04196530e-01],
[9.79657844e-01, 2.03421564e-02],
[4.24492959e-03, 9.95755070e-01],
[4.46627026e-01, 5.53372974e-01],
[1.04693540e-01, 8.95306460e-01],
[1.74855153e-01, 8.25144847e-01],
[6.03209371e-02, 9.39679063e-01],
[6.56148601e-02, 9.34385140e-01],
[1.19933445e-01, 8.80066555e-01],
[8.58104363e-03, 9.91418956e-01],
[8.59876891e-01, 1.40123109e-01],
[6.84717395e-01, 3.15282605e-01],
[1.95234632e-01, 8.04765368e-01],
[9.82841776e-01, 1.71582240e-02],
[9.47844801e-01, 5.21551992e-02],

[6.97641026e-04, 9.99302359e-01],
[3.92553985e-02, 9.60744601e-01],
[9.40310910e-01, 5.96890899e-02],
[2.89272173e-02, 9.71072783e-01],
[5.42272369e-02, 9.45772763e-01],
[1.63539867e-01, 8.36460133e-01],
[2.30256645e-03, 9.97697434e-01],
[1.31058827e-01, 8.68941173e-01],
[7.18464282e-02, 9.28153572e-01],
[5.19747785e-03, 9.94802522e-01],
[1.66743033e-01, 8.33256967e-01],
[8.64722657e-01, 1.35277343e-01],
[2.03828936e-01, 7.96171064e-01],
[3.85192279e-02, 9.61480772e-01],
[4.25737942e-01, 5.74262058e-01],
[1.22169118e-01, 8.77830882e-01],
[9.69781847e-01, 3.02181531e-02],
[8.44232927e-01, 1.55767073e-01],
[9.51011404e-01, 4.89885962e-02],
[3.65231656e-02, 9.63476834e-01],
[2.97233519e-01, 7.02766481e-01],
[9.83930559e-01, 1.60694414e-02],
[8.50482377e-01, 1.49517623e-01],
[9.69227146e-01, 3.07728545e-02],
[5.70768072e-01, 4.29231928e-01],
[6.08967868e-01, 3.91032132e-01],
[1.52018704e-03, 9.98479813e-01],
[9.05744695e-01, 9.42553048e-02],
[9.84757221e-01, 1.52427790e-02],
[9.88122823e-01, 1.18771771e-02],
[2.69225535e-02, 9.73077447e-01],
[9.93780846e-01, 6.21915371e-03],
[9.23251233e-01, 7.67487674e-02],
[4.95989754e-01, 5.04010246e-01],
[9.29589501e-01, 7.04104985e-02],
[9.83112237e-01, 1.68877626e-02],
[3.74683662e-03, 9.96253163e-01],
[9.41062433e-01, 5.89375673e-02],
[4.83198636e-01, 5.16801364e-01],
[1.28935351e-01, 8.71064649e-01],
[9.66520562e-01, 3.34794385e-02],
[9.51188169e-02, 9.04881183e-01],
[8.98677581e-01, 1.01322419e-01],
[9.34911237e-01, 6.50887635e-02],
[8.83531964e-01, 1.16468036e-01],
[7.75763221e-01, 2.24236779e-01],
[8.74341863e-01, 1.25658137e-01],
[8.47570117e-01, 1.52429883e-01],
[9.62574483e-01, 3.74255167e-02],
[2.72313989e-01, 7.27686011e-01],
[1.02516135e-01, 8.97483865e-01],
[5.00234285e-01, 4.99765715e-01],
[8.54019577e-01, 1.45980423e-01],
[9.18039310e-01, 8.19606897e-02],
[1.42416420e-01, 8.57583580e-01],
[5.78337026e-02, 9.42166297e-01],
[8.08102236e-01, 1.91897764e-01],
[5.20460632e-01, 4.79539368e-01],
[1.34058715e-01, 8.65941285e-01],
[9.08184651e-02, 9.09181535e-01],

[8.49719108e-03, 9.91502809e-01],
[2.60309646e-01, 7.39690354e-01],
[5.84141579e-02, 9.41585842e-01],
[3.63798304e-02, 9.63620170e-01],
[1.19264420e-01, 8.80735580e-01],
[7.89885122e-01, 2.10114878e-01],
[8.43222920e-01, 1.56777080e-01],
[9.98692380e-01, 1.30761976e-03],
[8.78874611e-01, 1.21125389e-01],
[3.56772596e-02, 9.64322740e-01],
[6.03050103e-01, 3.96949897e-01],
[9.86431515e-01, 1.35684853e-02],
[7.93146504e-01, 2.06853496e-01],
[8.47467088e-01, 1.52532912e-01],
[1.13676130e-01, 8.86323870e-01],
[8.19139947e-01, 1.80860053e-01],
[8.96156182e-01, 1.03843818e-01],
[9.08710566e-02, 9.09128943e-01],
[1.34428958e-02, 9.86557104e-01],
[6.36408287e-03, 9.93635917e-01],
[8.98000692e-01, 1.01999308e-01],
[1.42696817e-02, 9.85730318e-01],
[6.47047414e-01, 3.52952586e-01],
[6.45803708e-01, 3.54196292e-01],
[8.98928926e-01, 1.01071074e-01],
[3.98241682e-01, 6.01758318e-01],
[4.78051531e-02, 9.52194847e-01],
[1.25475872e-01, 8.74524128e-01],
[4.58967728e-04, 9.99541032e-01],
[1.81797283e-01, 8.18202717e-01],
[9.76613797e-01, 2.33862032e-02],
[9.05272249e-01, 9.47277509e-02],
[7.81829796e-01, 2.18170204e-01],
[9.35073716e-01, 6.49262838e-02],
[9.77100484e-01, 2.28995160e-02],
[4.05110751e-01, 5.94889249e-01],
[9.51057021e-01, 4.89429794e-02],
[1.38470656e-03, 9.98615293e-01],
[8.12430453e-01, 1.87569547e-01],
[2.25585075e-01, 7.74414925e-01],
[2.36371049e-01, 7.63628951e-01],
[8.08299504e-01, 1.91700496e-01],
[9.36603714e-01, 6.33962865e-02],
[1.88738512e-02, 9.81126149e-01],
[1.77463037e-01, 8.22536963e-01],
[2.46424378e-02, 9.75357562e-01],
[8.70661557e-01, 1.29338443e-01],
[9.63067534e-04, 9.99036932e-01],
[8.29856023e-02, 9.17014398e-01],
[9.78392538e-01, 2.16074617e-02],
[9.69095061e-01, 3.09049389e-02],
[7.81409729e-01, 2.18590271e-01],
[9.96733261e-01, 3.26673858e-03],
[9.57757229e-01, 4.22427714e-02],
[8.04243134e-01, 1.95756866e-01],
[9.61517017e-01, 3.84829830e-02],
[9.97874704e-01, 2.12529648e-03],
[9.24711398e-01, 7.52886020e-02],
[2.42338405e-02, 9.75766160e-01],
[1.35488530e-01, 8.64511470e-01],

[9.93125832e-01, 6.87416827e-03],
[3.44935605e-03, 9.96550644e-01],
[7.44660946e-01, 2.55339054e-01],
[1.36443551e-01, 8.63556449e-01],
[2.40974492e-02, 9.75902551e-01],
[8.43420401e-01, 1.56579599e-01],
[1.45786514e-04, 9.99854213e-01],
[9.40088052e-01, 5.99119484e-02],
[1.05122328e-02, 9.89487767e-01],
[7.86963112e-01, 2.13036888e-01],
[7.24006621e-02, 9.27599338e-01],
[4.59899501e-01, 5.40100499e-01],
[4.78127589e-01, 5.21872411e-01],
[9.84777169e-01, 1.52228310e-02],
[9.27074807e-01, 7.29251934e-02],
[6.48923088e-03, 9.93510769e-01],
[5.64510633e-02, 9.43548937e-01],
[3.32902371e-02, 9.66709763e-01],
[2.84420293e-03, 9.97155797e-01],
[9.84840147e-01, 1.51598529e-02],
[2.32767092e-01, 7.67232908e-01],
[5.81285578e-01, 4.18714422e-01],
[6.14511717e-02, 9.38548828e-01],
[2.20625875e-02, 9.77937413e-01],
[9.97792594e-01, 2.20740624e-03],
[9.69144758e-01, 3.08552418e-02],
[6.10729149e-01, 3.89270851e-01],
[4.52003638e-03, 9.95479964e-01],
[5.22476985e-03, 9.94775230e-01],
[8.38892074e-01, 1.61107926e-01],
[2.94088160e-02, 9.70591184e-01],
[2.44811806e-02, 9.75518819e-01],
[5.04717788e-01, 4.95282212e-01],
[9.96970998e-01, 3.02900221e-03],
[4.38357935e-01, 5.61642065e-01],
[7.55533719e-01, 2.44466281e-01],
[7.21767578e-01, 2.78232422e-01],
[9.15377123e-01, 8.46228773e-02],
[9.55609414e-01, 4.43905856e-02],
[1.49458769e-01, 8.50541231e-01],
[4.73409989e-01, 5.26590011e-01],
[8.82433027e-01, 1.17566973e-01],
[1.02705527e-02, 9.89729447e-01],
[9.86046455e-01, 1.39535451e-02],
[1.41031062e-01, 8.58968938e-01],
[1.44353766e-01, 8.55646234e-01],
[4.31374884e-04, 9.99568625e-01],
[8.21147498e-01, 1.78852502e-01],
[9.60790250e-01, 3.92097505e-02],
[7.05603626e-03, 9.92943964e-01],
[8.69523194e-01, 1.30476806e-01],
[4.76289789e-02, 9.52371021e-01],
[3.64451357e-01, 6.35548643e-01],
[7.16396395e-01, 2.83603605e-01],
[4.75930773e-01, 5.24069227e-01],
[2.76513494e-01, 7.23486506e-01],
[1.77143904e-01, 8.22856096e-01],
[3.87694387e-01, 6.12305613e-01],
[8.73586021e-02, 9.12641398e-01],
[8.74641664e-01, 1.25358336e-01]])


```
In [29]: from sklearn.model_selection import StratifiedKFold, GridSearchCV

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(
    estimator=model,
    param_grid=params,
    cv=cv,
    n_jobs=1
)
```

```
In [30]: print(params)

{'penalty': ['l1', 'l2', 'elasticnet'], 'C': [100, 10, 1.0, 0.1, 0.01], 'solver': ['newton_cg', 'lbfgs', 'liblinear', 'sag', 'saga']}
```

```
In [31]: grid
```

```
Out[31]:
```

```
  ▶ GridSearchCV ⓘ ?
  ▶ estimator: LogisticRegression
    ▶ LogisticRegression ?
```

```
In [32]: grid.fit(X_train, y_train)
```

```

~~~~~
File "/Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site
-packages/sklearn/linear_model/_logistic.py", line 1147, in fit
    solver = _check_solver(self.solver, penalty, self.dual)
~~~~~
File "/Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site
-packages/sklearn/linear_model/_logistic.py", line 65, in _check_solver
    raise ValueError(
ValueError: Solver sag supports only 'l2' or None penalties, got elasticne
t penalty.

warnings.warn(some_fits_failed_message, FitFailedWarning)
/Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-package
s/sklearn/model_selection/_search.py:1137: UserWarning: One or more of the
test scores are non-finite: [      nan      nan 0.86571429      nan
0.86571429      nan
0.86571429 0.86571429 0.86571429 0.86571429      nan      nan
      nan      nan 0.86571429      nan      nan 0.86428571
      nan 0.86428571      nan 0.86428571 0.86428571 0.86428571
0.86428571      nan      nan      nan      nan      nan 0.86428571
      nan      nan 0.86714286      nan 0.86714286      nan
0.86428571 0.86428571 0.86428571 0.86428571      nan      nan
      nan      nan 0.86428571      nan      nan 0.87285714
      nan 0.87285714      nan 0.86285714 0.86285714 0.86285714
0.86285714      nan      nan      nan      nan      nan 0.86285714
      nan      nan 0.87      nan      nan 0.87142857      nan
0.87      0.87142857 0.87      0.87      nan      nan
      nan      nan 0.87      ]
warnings.warn(
/Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-package
s/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was dep
recated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instea
d. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty
='l1', and C=np.inf instead of penalty=None.
warnings.warn(
/Users/mujeebmohammed/Documents/DS/Python/venv/lib/python3.11/site-package
s/sklearn/linear_model/_logistic.py:1160: UserWarning: Inconsistent value
s: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_rati
o only.
warnings.warn(

```

Out [32]:

```

  ▶ GridSearchCV ⓘ ?
  ▶ best_estimator_: LogisticRegression
    ▶ LogisticRegression ?

```

In [34]: `grid.best_params_`

Out [34]: `{'C': 0.1, 'penalty': 'l1', 'solver': 'liblinear'}`

In [35]: `grid.best_score_`

Out [35]: `np.float64(0.8728571428571428)`

In [36]: `grid.predict(X_test)`

```
Out[36]: array([0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0,
                1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0,
                0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1,
                0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0,
                0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1,
                1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0,
                1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0,
                0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1,
                0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1,
                1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1,
                1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1,
                0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1,
                0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1,
                1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0])
```

```
In [38]: score=accuracy_score(y_pred,y_test)
print(score)
print(classification_report(y_pred,y_test))
print(confusion_matrix(y_pred,y_test))
```

```
0.8466666666666667
```

	precision	recall	f1-score	support
0	0.87	0.80	0.84	147
1	0.82	0.89	0.86	153
accuracy			0.85	300
macro avg	0.85	0.85	0.85	300
weighted avg	0.85	0.85	0.85	300

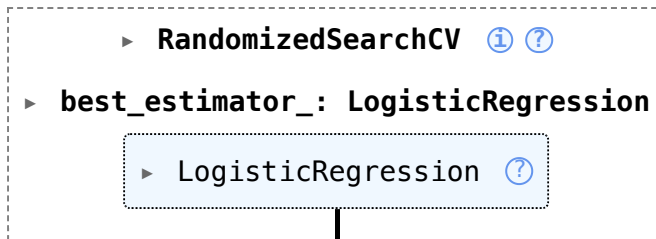
```
[[118 29]
 [ 17 136]]
```

```
In [41]: #Randomized Search CV

from sklearn.model_selection import RandomizedSearchCV
l=LogisticRegression()
randomcv=RandomizedSearchCV(estimator=model, param_distributions=params,
```

```
In [42]: randomcv.fit(X_train,y_train)
```

Out[42]:



In [44]: `randomcv.best_score_`

Out[44]: `np.float64(0.8785714285714287)`

In [45]: `randomcv.best_params_`

Out[45]: `{'solver': 'lbfgs', 'penalty': 'l2', 'C': 0.01}`

In [46]: `y_pred=randomcv.predict(X_test)`

In [47]: `score=accuracy_score(y_pred,y_test)`
`print(score)`
`print(classification_report(y_pred,y_test))`
`print(confusion_matrix(y_pred,y_test))`

0.8533333333333334

	precision	recall	f1-score	support
0	0.92	0.79	0.85	157
1	0.80	0.92	0.86	143
accuracy			0.85	300
macro avg	0.86	0.86	0.85	300
weighted avg	0.86	0.85	0.85	300

```
[[124  33]
 [ 11 132]]
```

In []: